ENGINEERING ESSENTIALS · FAANG INTERVIEW MASTER SERIES

Cloud & Infrastructure

Containers, Kubernetes, service mesh, CI/CD and the three pillars of observability.

Docker · Kubernetes · Service Mesh · CI/CD · IaC · Observability · SLI / SLO

VOLUME 04
THE CLOUD & INFRASTRUCTURE ISSUE

COMPREHENSIVE
DEEP DIVE + REVISION

LEVEL
SWE → STAFF

IN THIS ISSUE

Contents

SECTION 01

01 Overview — What It Is

Cloud & Infrastructure is the discipline of **packaging, deploying, operating, and observing software at scale** across pools of compute, network, and storage resources — without owning physical hardware.

The stack from atoms to apps:

CLOUD & INFRASTRUCTURE STACK

Infrastructure as Code	Terraform · CloudFormation · Pulumi
Observability	Metrics · Logs · Traces
CI/CD Pipelines	GitHub Actions · ArgoCD · Spinnaker
Service Mesh	Istio · Envoy · Linkerd
Container Orchestration	Kubernetes · ECS · Nomad
Virtual Machines / Containers	Docker · containerd · runc
Hypervisor / OS Kernel	KVM · VMware · Host Linux kernel
Physical Hardware	CPU · RAM · NIC · bare metal

Each layer abstracts complexity from the one below — containers share the kernel, VMs each carry their own OS.

Key players: **AWS, GCP, Azure** (IaaS/PaaS/SaaS providers), **Docker** (container runtime), **Kubernetes** (orchestrator), **Istio/Envoy** (service mesh), **Terraform** (IaC), **Prometheus/Grafana** (observability).

SECTION 02

02 Why It Exists

The Pre-Cloud Problem

Before containers and cloud:

- **Snowflake servers** — each machine configured by hand; "works on my machine" bugs everywhere.
- **Long provisioning cycles** — weeks to get a server racked, cabled, and OS-installed.
- **Resource waste** — typical server utilization: 5–15 %; paying for 100 %.
- **Deployment fear** — big-bang releases every 6 months; rollback meant physical disk swaps.
- **On-call nightmares** — no standardized way to observe what was broken.

The Solutions That Emerged

Problem	Solution
Works on my machine	Containers (immutable, portable environment)
Slow provisioning	Cloud APIs + IaC (seconds, not weeks)
Resource waste	Bin-packing orchestration (Kubernetes scheduling)
Deployment fear	CI/CD + progressive delivery (canary, blue-green)
Observability gap	Metrics, structured logs, distributed traces
Config drift	GitOps + Infrastructure as Code

SECTION 03

03 Why FAANG Cares (Company-Specific)

Google

- **Invented the paradigm.** Borg (2003) is Kubernetes' direct ancestor. Google ran everything in containers a decade before the public heard of Docker. They open-sourced Kubernetes in 2014 partly to commoditize AWS's lead.
- Interviews: expect deep K8s internals questions ("explain the scheduler"), Borg vs K8s design decisions, SRE concepts (SLO/error budget — Google coined them).

Meta (Facebook)

- Runs one of the world's largest bare-metal fleets; heavy investment in **Tupperware** (internal K8s-like system) and **Twine**.
- Cares about: resource efficiency, tens of thousands of services, deployment velocity at scale, Thrift/Proxygen service meshes.
- Interview focus: large-scale distributed systems, custom orchestration trade-offs.

Amazon (AWS)

- Cloud is their core business; they expect engineers to know AWS services deeply (ECS, EKS, Lambda, CloudFormation).
- Strong focus on **operational excellence** (first pillar of AWS Well-Architected Framework).
- Interview focus: IaC (CloudFormation/CDK), service reliability, cost optimization.

Apple

- Massive on-prem + private cloud (iCloud). K8s adoption is growing internally.
- Prioritizes **security** and **privacy** in infrastructure decisions.

Netflix

- Pioneered **chaos engineering** (Chaos Monkey), microservices on AWS, canary deployments, and spinnaker (open-sourced CI/CD).
- Interview focus: operational resilience, observability at scale, progressive delivery.

Interview Takeaway: K8s = Google's gift from Borg. SRE/SLO = Google concept. Chaos engineering = Netflix. AWS Well-Architected = Amazon.

SECTION 04

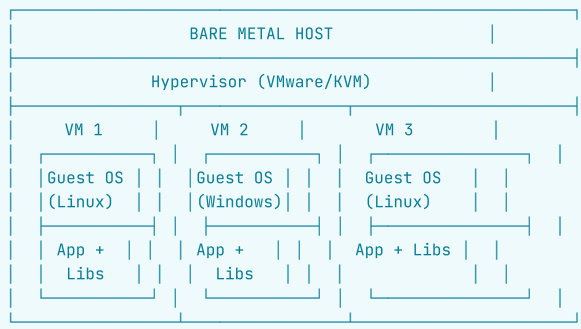
04 Core Concepts

Containers vs Virtual Machines

Virtual Machines — First Principles

A **hypervisor** runs on bare metal and creates isolated virtual hardware environments. Each VM gets its own kernel, OS, and binaries.

DIAGRAM

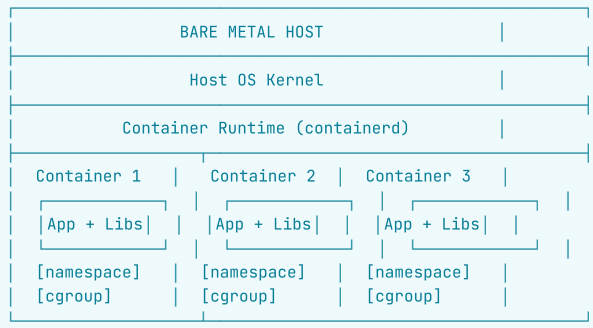


- **Hypervisor types:** Type 1 (bare metal — VMware ESXi, KVM, Hyper-V); Type 2 (runs on OS — VirtualBox, VMware Workstation).
- **Isolation:** strongest — separate kernel, separate memory space.
- **Boot time:** 30 seconds – 5 minutes.
- **Size:** GBs (full OS included).

Containers — First Principles

Containers share the **host OS kernel** but isolate processes using two Linux kernel features: **namespaces** and **cgroups**.

DIAGRAM



- **Boot time:** milliseconds (process, not OS).
- **Size:** MBs (shares kernel; only app + libs).
- **Isolation:** weaker than VMs (kernel shared); container escape = privilege escalation on host.

Containers vs VMs — Full Comparison

Dimension	Containers	Virtual Machines
Isolation unit	Process group (NS + cgroup)	Full OS + virtual hardware
Kernel	Shared with host	Own kernel per VM
Boot time	Milliseconds	Seconds to minutes
Image size	MBs	GBs
Resource overhead	Near-zero (no guest OS)	High (guest OS + hypervisor)
Security isolation	Weaker (kernel shared)	Stronger (full isolation)
Portability	Excellent (OCI standard)	Good (VM images are large)
Density (per host)	100s of containers	10s of VMs
Use case	Microservices, CI/CD	Multi-tenant, legacy lift-and-shift

Container Internals

Linux Namespaces

Namespaces partition Linux kernel resources so each container sees its own isolated view.

Namespace	Isolates	Example
pid	Process IDs	PID 1 inside container ≠ PID 1 on host
net	Network interfaces, routing tables	Container gets its own `eth0`
mnt	Filesystem mount points	Container's `/` is its own root
uts	Hostname and domain name	`hostname` inside = "my-container"
ipc	IPC (semaphores, shared mem)	No inter-container IPC by default
user	User/Group IDs	UID 0 in container ≠ UID 0 on host
cgroup (v2)	cgroup root	Nested cgroup visibility
time (Linux 5.6+)	System clock	Different clock offset per container

How to verify: `ls -la /proc/self/ns/` on any Linux machine shows your current namespaces.

Control Groups (cgroups)

cgroups **limit and account for** resource usage. Without cgroups, one container could starve all others.

Resources controlled by cgroups: - **CPU:** cpu.shares, cpu.quota/period (CFS bandwidth) - **Memory:** memory.limit_in_bytes, memory.memsw (swap) - **Block I/O:** blkio.weight, read/write BPS limits - **Network:** (tc/netfilter, not cgroup directly) - **PIDs:** pids.max (prevent fork bombs)

cgroup hierarchy:

DIAGRAM

```
/sys/fs/cgroup/
  ├── cpu/
  │   └── docker/
  │       └── <container-id>/
  │           ├── cpu.shares       ← relative weight
  │           └── cpu.cfs_quota_us ← hard limit
  └── memory/
      └── docker/
          └── <container-id>/
              └── memory.limit_in_bytes
```

Interview Takeaway: Namespaces = what you can see. cgroups = what you can use.

Union Filesystems (OverlayFS)

Docker images are **layered**. Each layer is a diff (like a git commit). At runtime, layers are stacked using **OverlayFS** (or older: AUFS, devicemapper).

DOCKER IMAGE LAYERS · OVERLAYFS

R/W	**Container Layer**	ephemeral — lost on container stop
4	**COPY app/ /app**	image layer (read-only)
3	**RUN pip install**	image layer (read-only)
2	**RUN apt-get update**	image layer (read-only)
1	**FROM ubuntu:22.04**	base image layer (read-only, shared on disk)

OverlayFS merges all read-only layers + writable container layer into a single unified view — unchanged layers are shared across containers.

OverlayFS terms: - **lowerdir:** read-only image layers (stacked) - **upperdir:** read-write container layer - **workdir:** OverlayFS internal scratch space - **merged:** unified view the container sees

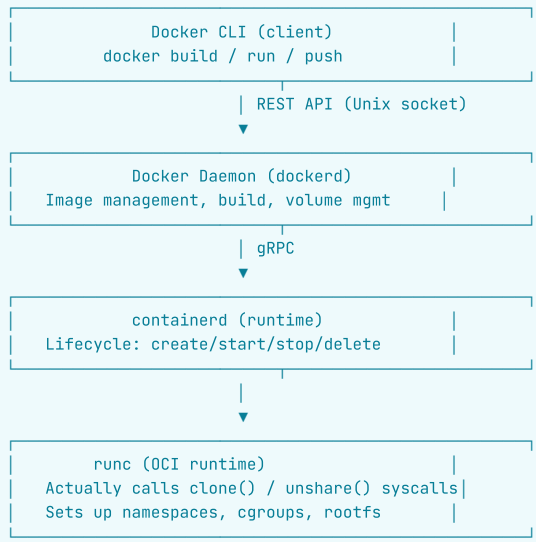
Copy-on-Write (CoW): When a container modifies a file from a lower layer, OverlayFS copies it to upperdir first, then modifies. Original lower layer is untouched → other containers sharing that layer are unaffected.

Why this matters: Multiple containers share base layers on disk. A 500 MB base image shared by 50 containers costs 500 MB + 50 × (small diff), not 50 × 500 MB.

Docker Deep Dive

Docker Architecture

DIAGRAM



OCI (Open Container Initiative): Standardizes image format and runtime spec. Any OCI-compliant runtime (runc, crun, gVisor's runsc) can run OCI images.

Dockerfile Best Practices

```dockerfile
# GOOD: Order layers least-to-most-frequently-changed
FROM python:3.11-slim          # base layer (rarely changes)
WORKDIR /app
COPY requirements.txt .        # deps layer (changes when deps change)
RUN pip install --no-cache-dir -r requirements.txt
COPY . .                       # app layer (changes every build)

# Multi-stage build: don't ship build tools in prod image
FROM python:3.11-slim AS builder
RUN pip install build
COPY . .
RUN python -m build

FROM python:3.11-slim AS runtime
COPY --from=builder /app/dist /app/dist
```

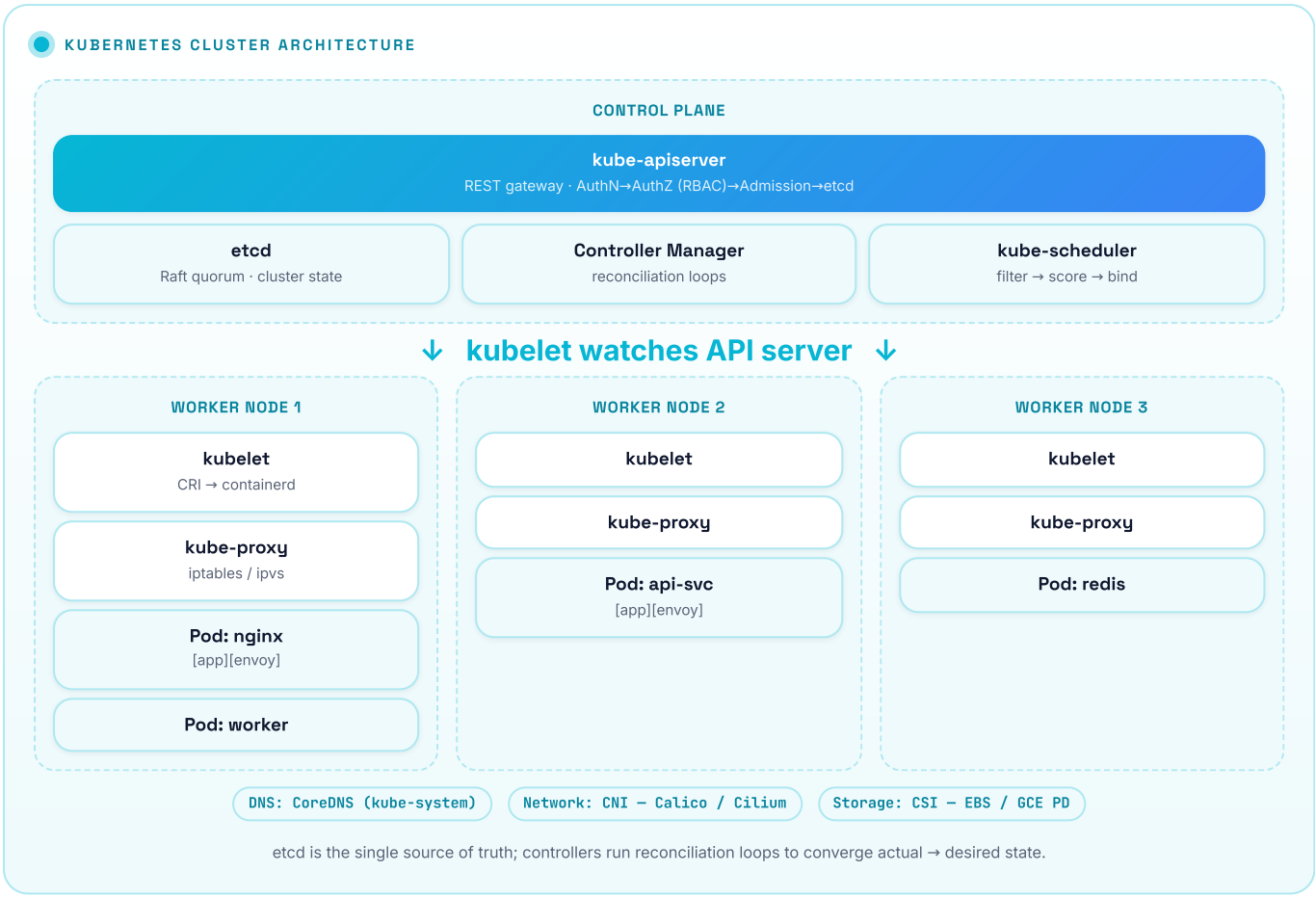
Key practices: - **Minimize layers:** chain RUN commands with && - **Order for cache:** put stable layers first - **Multi-stage builds:** build in fat image, copy artifacts to slim runtime image - **Non-root user:** USER `appuser` for least privilege - **Pin base image tags:** `python:3.11.5-slim` not `python:latest` - **.dockerignore:** exclude `.git`, `node_modules`, test files

Docker Networking

Network Mode	Description	Use Case
bridge (default)	Software bridge; containers get IPs in 172.17.0.0/16; NAT to host	Single-host development
host	Container shares host network namespace	High-performance, port 80 = host port 80
overlay	Multi-host networking (Swarm/K8s)	Distributed clusters
none	No networking	Air-gapped processing jobs
macvlan	Container appears as physical NIC on LAN	Network appliances

Kubernetes Architecture

The Big Picture



Control Plane Components

1. kube-apiserver - The **single entry point** for all cluster operations (kubectl, controllers, kubelets all talk to it). - Validates and persists resource state to etcd. - Exposes REST API; implements authentication, authorization (RBAC), admission control. - Stateless — can run multiple replicas behind a load balancer.

2. etcd - Distributed key-value store; source of truth for all cluster state. - Uses **Raft consensus** — writes must be acknowledged by majority of nodes. - Only the API server talks to etcd (not kubelets, not controllers directly). - **Catastrophic if lost** — etcd backup is the most critical operational task. - Keys look like: `/registry/pods/default/my-pod`

3. kube-scheduler - Watches for unscheduled pods (pods with no `nodeName` set). - **Two phases:** Filtering (which nodes can run this pod?) → Scoring (which node is best?). - Filtering criteria: resource requests fit, node selectors match, taints/tolerations, affinity/anti-affinity, PodTopologySpread. - Scoring criteria: LeastRequestedPriority, BalancedResourceAllocation, ImageLocality. - Writes the winning `nodeName` back to the pod spec via API server.

4. kube-controller-manager - Runs many independent **control loops** in a single process. - Each controller watches desired state (spec) vs actual state (status) and reconciles.

Controller	What it manages
ReplicaSet controller	Ensures N pod replicas running
Deployment controller	Manages ReplicaSets for rolling updates
Node controller	Detects/marks NotReady nodes, evicts pods
Job controller	Runs pods to completion
Endpoint controller	Populates Endpoints objects for Services
ServiceAccount controller	Creates default SAs in new namespaces
Namespace controller	Handles namespace deletion cleanup

5. cloud-controller-manager - Splits cloud-specific logic out of kube-controller-manager. - Manages: cloud load balancers (for Services type=LoadBalancer), cloud routes, cloud volumes.

Worker Node Components

kubelet - Agent running on every node. Talks to API server, ensures pods are running. - Receives a PodSpec, calls the container runtime (via CRI — Container Runtime Interface) to start containers. - Reports pod status, node conditions (memory pressure, disk pressure) back to API server. - Runs liveness/readiness probes and restarts failing containers.

kube-proxy - Implements the `Service` abstraction: translates ClusterIP/NodePort to actual pod IPs. - Modes: **iptables** (default — installs iptables rules), **ipvs** (faster, handles 10k+ services), **eBPF** (Cilium — bypass kernel networking stack). - Does NOT proxy traffic itself in iptables mode; sets up rules so kernel routes directly.

Container Runtime Interface (CRI) - kubelet talks CRI; any conformant runtime works: containerd (default), CRI-O, Docker (via dockershim — deprecated K8s 1.24).

Kubernetes Workload Resources

Pod

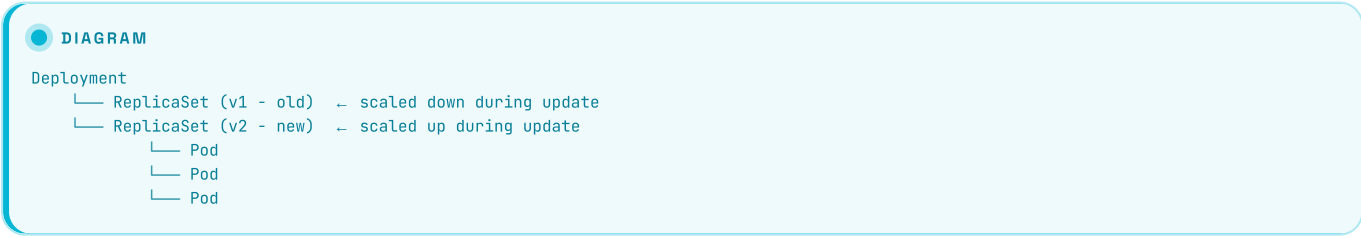
Smallest deployable unit. One or more containers sharing: - **Network namespace** (same IP, can communicate via localhost) - **IPC namespace** - **Volumes** (mounted into each container independently)

```yaml
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: app
    image: myapp:1.0
    resources:
      requests:          # used for scheduling decisions
        cpu: "250m"      # 250 milliCPU = 0.25 cores
        memory: "256Mi"
      limits:            # hard caps enforced by cgroups
        cpu: "500m"
        memory: "512Mi"
    readinessProbe:      # when to send traffic
      httpGet: {path: /health, port: 8080}
      initialDelaySeconds: 5
    livenessProbe:       # when to restart
      httpGet: {path: /health, port: 8080}
      failureThreshold: 3
```

Requests vs Limits: - `requests` = what the scheduler uses to find a node with enough capacity. - `limits` = cgroup enforcement at runtime. CPU limit = throttling. Memory limit = OOMKill. - **Best practice:** always set requests; set memory limits; be careful with CPU limits (throttling causes latency, not kill).

Deployment

Manages ReplicaSets for rolling updates and rollbacks.



Update strategy: `RollingUpdate` with `maxSurge` and `maxUnavailable` controls.

Services

Stable virtual IP + DNS name in front of a dynamic set of pods (selected by label selectors).

Service Type	Description	Use Case
ClusterIP	Virtual IP accessible only within cluster	Internal microservice communication
NodePort	Exposes service on every node's IP at static port (30000-32767)	External access in dev/bare-metal
LoadBalancer	Provisions cloud LB; external IP	Production external access on cloud
ExternalName	CNAME alias to external DNS	Route traffic to external DB
Headless (ClusterIP: None)	Returns pod IPs directly, no VIP	StatefulSets, service discovery by DNS

Ingress

Layer 7 (HTTP/HTTPS) routing into the cluster. Requires an **Ingress Controller** (nginx-ingress, AWS ALB Ingress, Traefik).

DIAGRAM

```
Internet → LoadBalancer → Ingress Controller → Service → Pods
                          (nginx pod)
                          Routes by:
                          - hostname (api.example.com)
                          - path (/api/v1, /static)
                          - TLS termination
```

ConfigMaps and Secrets

Resource	Purpose	Storage	Encoding
ConfigMap	Non-sensitive config (env vars, config files)	etcd plaintext	None
Secret	Sensitive data (passwords, tokens, TLS certs)	etcd base64	base64 (NOT encryption by default!)

Interview Gotcha: Secrets are base64-encoded, NOT encrypted by default. Enable **encryption at rest** in etcd (`EncryptionConfiguration`) and use external secret managers (Vault, AWS Secrets Manager) for production.

Consumption methods: - **Environment variables:** `envFrom: [configMapRef, secretRef]` - **Volume mounts:** mounted as files (auto-updates when CM/Secret changes)

StatefulSets

For stateful apps (databases, Kafka, ZooKeeper). Provides: - **Stable pod identity:** `pod-0`, `pod-1`, `pod-2` (not random hash names) - **Stable network identity:** `pod-0.svc.namespace.svc.cluster.local` - **Ordered deployment/scaling:** pod-0 must be Running before pod-1 starts - **Persistent Volume Claims per pod:** each pod gets its own PVC (not shared)

DaemonSet

Ensures one pod runs on **every node** (or nodes matching a nodeSelector). Use cases: log collectors (Fluentd), metrics agents (node-exporter), network plugins (Calico, Cilium).

Jobs and CronJobs

- **Job:** runs pods to completion (batch processing, DB migrations). Retries on failure.
- **CronJob:** schedules Jobs on cron schedule. Creates a Job object on schedule.

Kubernetes Scheduling and Autoscaling

Scheduling Deep Dive

The scheduler's decision tree:

DIAGRAM

```
1. FilterPhase (eliminate nodes):
   ├── NodeUnschedulable (node cordoned?)
   ├── PodFitsResources (enough CPU/memory?)
   ├── MatchNodeSelector (labels match?)
   ├── NoDiskConflict (volume conflicts?)
   ├── NoVolumeZoneConflict (zone of PV matches node zone?)
   ├── CheckTolerations (pod tolerates node taints?)
   └── CheckNodeAffinity (pod's nodeAffinity rules?)

2. ScorePhase (rank remaining nodes):
   ├── LeastAllocated (prefer less-loaded nodes)
   ├── BalancedResourceAllocation (CPU/mem balance)
   ├── NodeAffinityPriority (preferred nodeAffinity score)
   ├── PodAffinityPriority (co-locate with related pods)
   ├── InterPodAntiAffinity (spread apart)
   └── ImageLocality (node already has image cached)

3. Bind: scheduler writes nodeName to pod via API server
```

Taints and Tolerations: - **Taint** on node: "don't schedule here unless you tolerate this" - **Toleration** on pod: "I can tolerate that taint" - Effects: `NoSchedule` (don't place new pods), `PreferNoSchedule` (soft), `NoExecute` (evict existing pods)

DIAGRAM

```
# Taint a node for GPU workloads only
kubectl taint nodes gpu-node-1 hardware=gpu:NoSchedule

# Pod that tolerates it
tolerations:
- key: "hardware"
  operator: "Equal"
  value: "gpu"
  effect: "NoSchedule"
```

Pod Affinity/Anti-Affinity:

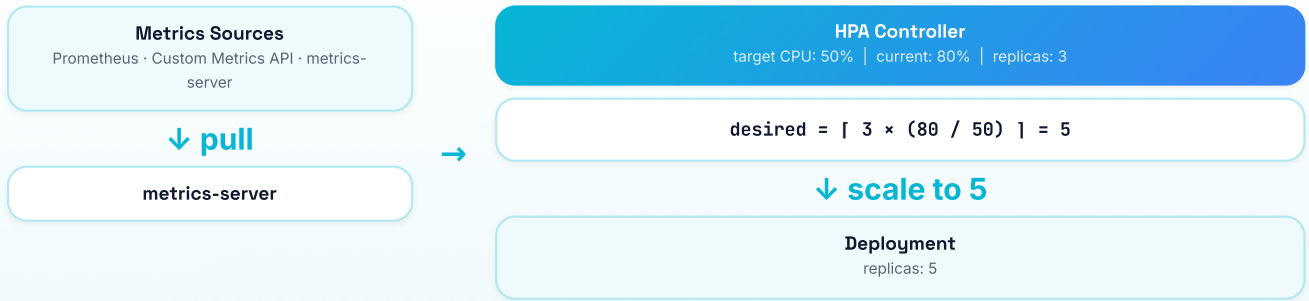
YAML

```yaml
affinity:
  podAntiAffinity:              # spread across zones
    requiredDuringScheduling...:
    - labelSelector:
        matchLabels: {app: my-api}
      topologyKey: topology.kubernetes.io/zone
```

Horizontal Pod Autoscaler (HPA)

Scales **number of pod replicas** based on metrics.

HORIZONTAL POD AUTOSCALER (HPA)



HPA polls metrics every 15 s; VPA adjusts resource requests; CA adds/removes nodes when pods stay unschedulable.

- Built-in metrics: CPU utilization, memory utilization (via metrics-server).
- Custom metrics: via Prometheus Adapter, KEDA (event-driven — queue depth, SQS messages).
- Scale-up: responsive (default: 3 min stabilization window).
- Scale-down: conservative (default: 5 min) to prevent flapping.

Vertical Pod Autoscaler (VPA)

Adjusts **CPU/memory requests and limits** per pod (not replica count). Requires pod restart (no in-place update until VPA v1 goes GA). Use HPA for scaling, VPA for right-sizing.

Cluster Autoscaler

Scales **number of nodes** in the cluster.

DIAGRAM

```
Pod pending (no node with enough resources)
        ↓
Cluster Autoscaler detects unschedulable pods
        ↓
Calls cloud API: add node to node group
        ↓
New node registers with kubelet → scheduler places pod
```

Scale-down: node is underutilized (<50% requested) for 10+ minutes → drain → terminate.

Interview Takeaway: HPA scales pods. VPA sizes pods. Cluster Autoscaler scales nodes. Use HPA + Cluster Autoscaler together; VPA conflicts with HPA on CPU/memory.

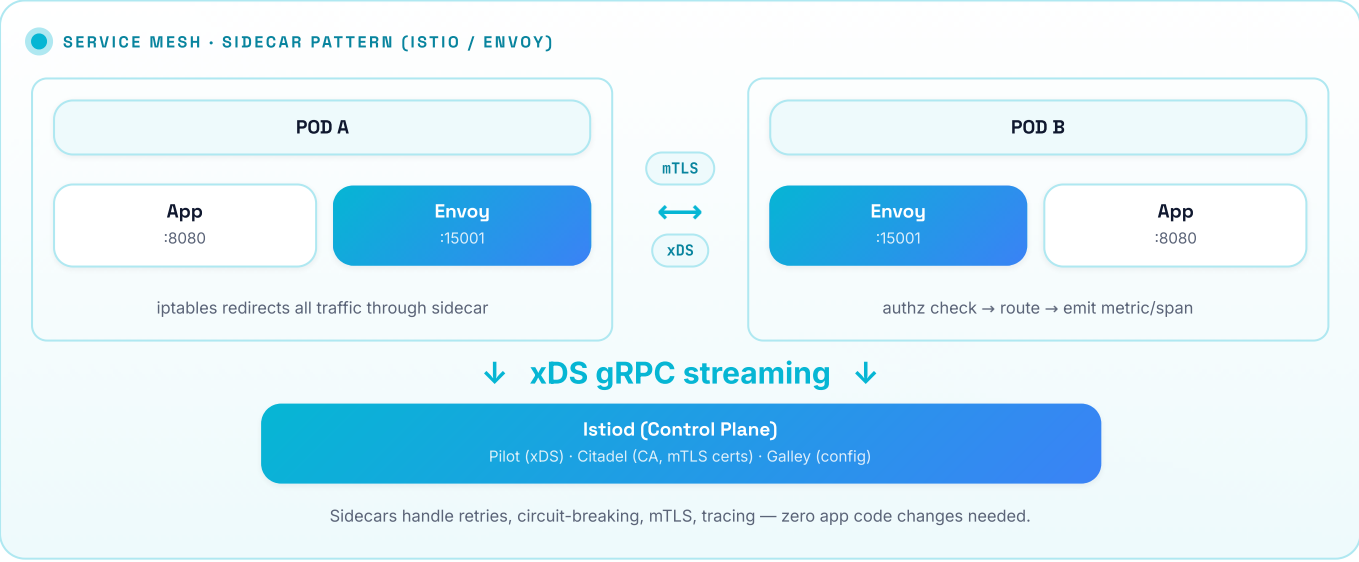
Service Mesh (Istio / Envoy)

Why Service Meshes Exist

In microservice architectures with 100s of services, cross-cutting concerns repeat in every service: - mTLS between services (auth + encryption) - Retry logic, circuit breaking, timeouts - Load balancing (layer 7 — weighted, consistent hash) - Observability (distributed tracing per request) - Traffic splitting (canary deployments)

Without mesh: each team reimplements in their SDK/library (fragile, polyglot nightmare). With mesh: **extracted to infrastructure layer** — language-agnostic, zero code changes.

Sidecar Pattern



Traffic Interception: Istio uses `istio-init` init container to install iptables rules that redirect all inbound/outbound traffic through Envoy on port 15001/15006. The app itself is completely unaware.

Envoy Proxy Internals

Envoy is the data-plane component (the actual sidecar). Key concepts:

- **Listeners:** what ports to accept connections on
- **Routes:** URL/header matching → cluster selection
- **Clusters:** upstream service + load balancing policy
- **Endpoints:** actual IP:port of upstream pods (populated via EDS — Endpoint Discovery Service)

xDS API: Envoy connects to control plane (Istiod) and dynamically receives updates for Listeners, Routes, Clusters, Endpoints without restarts.

Istio Features

Feature	How It Works
mTLS	Istiod issues X.509 certs (SPIFFE IDs); Envoy does TLS handshake transparently
Traffic splitting	VirtualService: 90% → v1, 10% → v2
Circuit breaking	OutlierDetection: eject pod after N consecutive 5xx errors
Retries	VirtualService: retry on 5xx, up to 3 attempts
Fault injection	Testing: inject 500ms delay or HTTP 503 to service
Observability	Envoy emits metrics/traces to Prometheus/Jaeger automatically
Ingress Gateway	Envoy at cluster edge (replaces Ingress for L7)

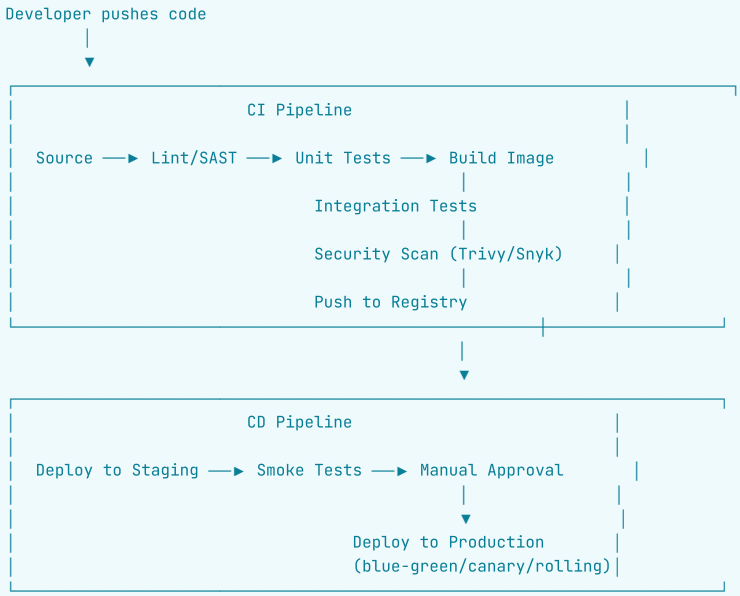
CI/CD Pipelines

What CI/CD Means

- **CI (Continuous Integration):** Every code change triggers automated build + test. Catches bugs early, keeps main branch deployable.
- **CD (Continuous Delivery):** Artifact is automatically deployable to production (but may require manual approval trigger).
- **CD (Continuous Deployment):** Every passing change automatically deploys to production (no manual gate).

Typical CI/CD Pipeline

DIAGRAM



Tools: GitHub Actions, GitLab CI, Jenkins, CircleCI, Tekton (K8s-native), ArgoCD (GitOps CD).

Deployment Strategies

Blue-green = instant cutover; rolling = gradual in-place; canary = risk-gated traffic shift with real-user validation.

Strategy	Rollback Speed	Cost	Downtime	Use Case
Blue-Green	Instant (LB flip)	2× infrastructure	Zero	Mission-critical, scheduled maintenance
Canary	Fast (reduce %)	~10% extra	Zero	Risk-sensitive, gradual validation
Rolling	Slow (re-roll)	No extra (in-place)	Zero (if configured)	Default K8s, resource-constrained
Recreate	N/A	Minimal	Yes!	Dev environments, stateful where overlap forbidden

Interview Takeaway: Canary = validate on real traffic subset. Blue-green = instant switchover. Rolling = gradual in-place. FAANG uses canary most for production.

GitOps

Git is the single source of truth for desired cluster state. CD tool (ArgoCD, Flux) watches git repo and syncs cluster to match.

DIAGRAM

```
Dev pushes Helm chart / K8s manifests to Git
        |
        ▼
ArgoCD detects diff (desired ≠ actual)
        |
        ▼
ArgoCD applies manifests to cluster
        |
        ▼
Cluster reaches desired state
```

Benefits: Auditability (git log = deployment log), easy rollback (git revert), separation of CI (build) and CD (deploy).

Infrastructure as Code

Why IaC

Manual cloud console clicks: not reproducible, not reviewable, not auditable, can't be version-controlled. IaC solves all four.

Terraform

HashiCorp's **cloud-agnostic** IaC tool using HCL (HashiCorp Configuration Language).

Core Workflow:

DIAGRAM

```
terraform init     → download providers, set up backend
terraform plan     → show what WILL change (dry run)
terraform apply    → make changes (asks confirm)
terraform destroy  → tear everything down
```

State file (`terraform.tfstate`): - JSON file mapping Terraform resources to real cloud resources. - MUST be stored remotely (S3 + DynamoDB for locking) for teams. - Sensitive data in state! Encrypt at rest.

Key Concepts: - **Providers:** plugins for AWS, GCP, Azure, Kubernetes, etc. - **Resources:** individual infrastructure objects (`aws_instance`, `google_container_cluster`). - **Data sources:** read existing resources (not manage them). - **Modules:** reusable, composable groups of resources (like functions). - **Workspace:** multiple state files from same config (dev/staging/prod). - **Remote state:** `terraform_remote_state` data source to read outputs from other stacks.

```hcl
# Example: EKS cluster
resource "aws_eks_cluster" "main" {
  name     = var.cluster_name
  role_arn = aws_iam_role.cluster.arn
  version  = "1.28"

  vpc_config {
    subnet_ids = var.private_subnet_ids
  }
}
```

Terraform vs CloudFormation:

Dimension	Terraform	CloudFormation
Cloud support	Multi-cloud	AWS only
Language	HCL (human-friendly)	JSON/YAML (verbose)
State management	External state file (S3 + lock)	Managed by AWS (no state file to manage)
Drift detection	`terraform plan`	CloudFormation drift detection
Rollback	Manual (or Terraform Cloud)	Automatic on stack failure
Ecosystem	Vast module registry	AWS-native, CDK wraps it
CDK equivalent	CDK for Terraform (CDKTF)	AWS CDK (TypeScript/Python → CFN)

Interview Takeaway: Terraform = multi-cloud, HCL, manage your own state. CloudFormation = AWS-only, native integration, AWS manages state.

Pulumi

Code-first IaC (TypeScript, Python, Go) — same model as Terraform but uses real programming languages. Good for complex conditional logic.

Ansible

Configuration management (not primarily resource provisioning). Agentless, SSH-based, idempotent playbooks. Good for OS-level config, not cloud resource lifecycle.

Monitoring, Logging, and Observability

The Three Pillars of Observability

OBSERVABILITY · THREE PILLARS

METRICS	LOGS	TRACES
What happened (aggregated numbers)	Why it happened (event detail)	Where time was spent across services
Prometheus	ELK Stack	Jaeger / Zipkin
Grafana	Loki	AWS X-Ray
Datadog	CloudWatch / Splunk	OpenTelemetry

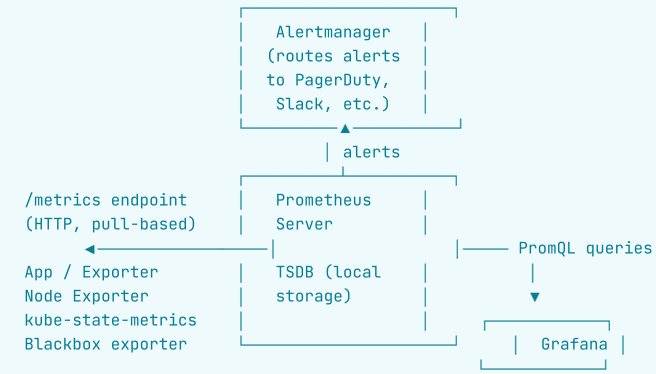
Metrics say *something is wrong*, logs say *why*, traces say *where in the call graph* — all three needed for full observability.

Pillar	What It Is	When To Use	Cost Profile
Metrics	Numeric time-series (CPU %, req/s, error rate)	Alerting, dashboards, SLO tracking	Low (aggregated)
Logs	Timestamped text/JSON events	Debugging specific errors, audit trails	High (volume = cost)
Traces	End-to-end request path across services with timing	Diagnosing latency, finding bottlenecks	Medium

They are complementary, not competing. Alert on metric → look at logs → follow trace to find root cause.

Prometheus Architecture

DIAGRAM



Pull vs Push model: - Prometheus **pulls** metrics from targets at scrape_interval (default 15s). - **Pushgateway** for short-lived jobs that can't be scraped. - **Advantages of pull:** easier to see if target is down (scrape fails), simpler config (targets defined in Prometheus, not in app).

PromQL examples:

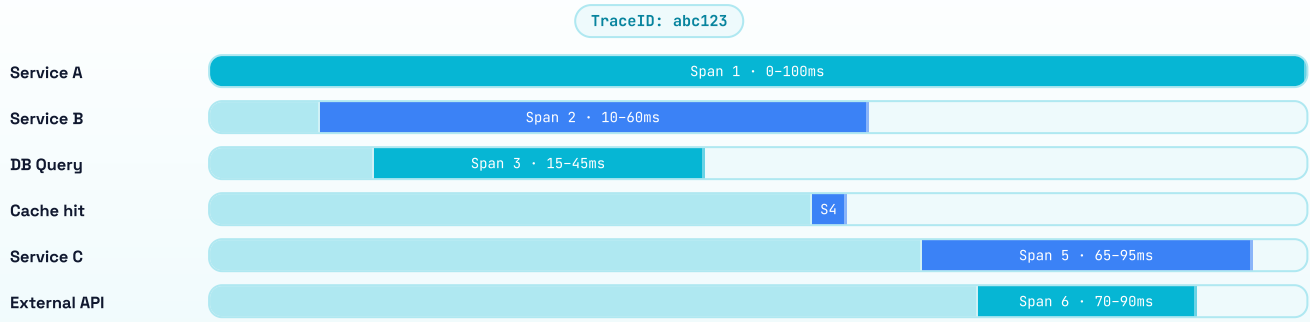
PROMQL

```promql
# Request rate over 5m
rate(http_requests_total[5m])

# 99th percentile latency
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))

# Error rate
sum(rate(http_requests_total{status=~"5.."}[5m]))
/ sum(rate(http_requests_total[5m]))
```

Metric types: | Type | Description | Example | |---|---|---| | **Counter** | Monotonically increasing; never decreases | `http_requests_total` | | **Gauge** | Up and down; current value | `memory_usage_bytes` | | **Histogram** | Samples in predefined buckets; compute percentiles | `http_request_duration_seconds_bucket` | | **Summary** | Pre-computed quantiles at scrape time | (less flexible, avoid) |

Distributed Tracing

DISTRIBUTED TRACING · WATERFALL VIEW



Each span records service name, start/end time, parent span ID — Jaeger / Tempo reconstruct the call graph and show the critical path.

OpenTelemetry (OTel): Vendor-neutral standard for instrumentation (SDK + API). Replaces vendor-specific clients. Exporters to Jaeger, Zipkin, Datadog, etc.

Sampling strategies: - **Head-based sampling:** decision at trace start (fast, simple, may miss rare errors). - **Tail-based sampling:** collect all spans, decide at trace end (catches all errors, expensive).

Structured Logging

JSON

```
{
  "timestamp": "2026-06-14T10:30:00Z",
  "level": "ERROR",
  "service": "payment-service",
  "trace_id": "abc123",
  "span_id": "def456",
  "user_id": "u-789",
  "message": "Payment failed: insufficient funds",
  "amount": 99.99,
  "currency": "USD"
}
```

Structured (JSON) logs enable: fast indexing, efficient queries, correlation with traces (via trace_id field).

Log aggregation pipeline:

DIAGRAM

App → Fluentd/Vector/Filebeat → Kafka (buffer) → Elasticsearch/Loki → Kibana/Grafana

SLI / SLO / SLA / Error Budgets

Definitions

Term	What It Is	Example
SLI (Service Level Indicator)	A metric measuring service behavior	"99th percentile latency of /api/checkout"
SLO (Service Level Objective)	Target value for an SLI	"p99 latency < 200ms for 99.9% of the time over 30 days"
SLA (Service Level Agreement)	Legal contract with customer; penalty for breach	"99.9% uptime; SLA breach → 10% service credit"
Error Budget	Allowed failure before SLO breach	"0.1% of requests can fail = 43.2 min/month"

The relationship:

DIAGRAM

```
SLI is measured
    ↓
Compared against SLO target
    ↓
Burns Error Budget when below target
    ↓
If budget exhausted → freeze new deployments, focus on reliability
    ↓
SLA is the external commitment (SLO with teeth)
```

Common SLIs

Service Type	Good SLI candidates
Web API	Availability (% 2xx), latency (p50/p99), error rate
Data pipeline	Freshness (lag), completeness (% records processed), throughput
Storage	Durability (% data retained), read latency
Streaming	Consumer lag, throughput (messages/s)

Error Budget Math

DIAGRAM

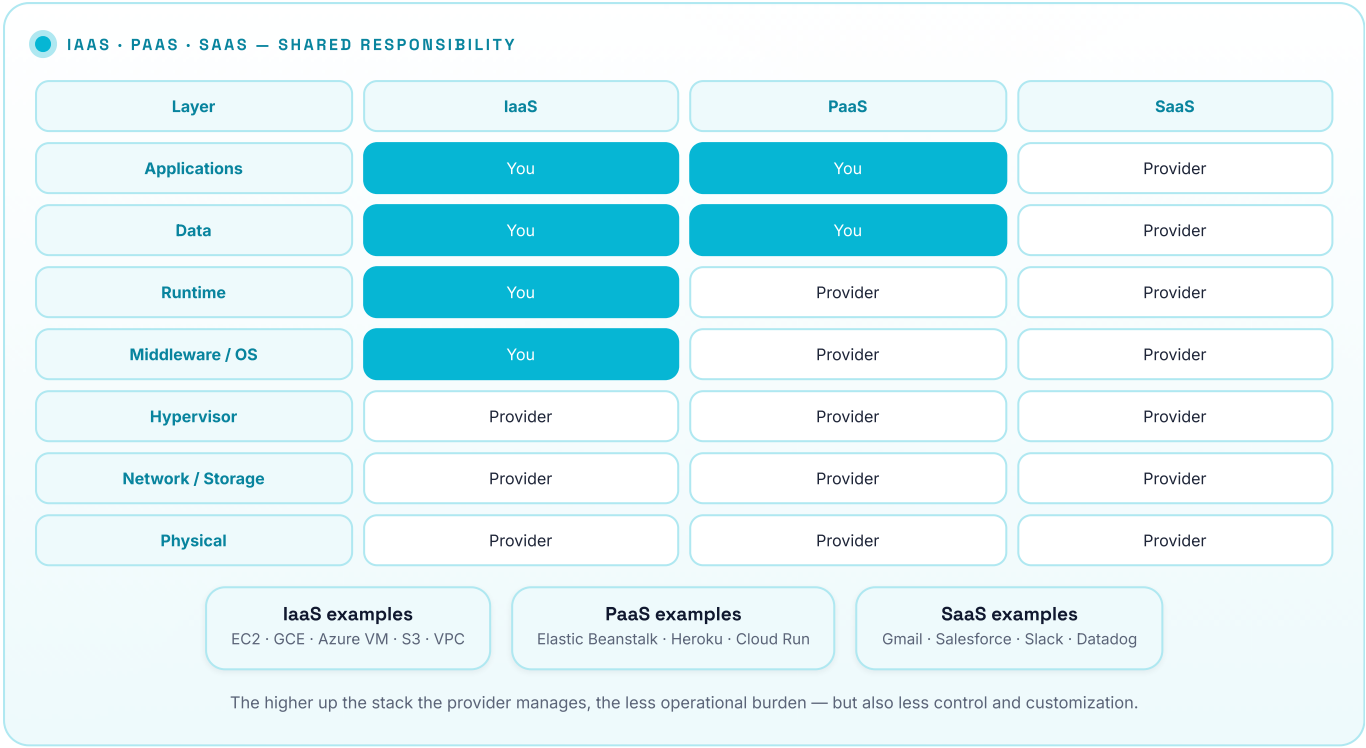
```
SLO: 99.9% availability
Error budget: 100% - 99.9% = 0.1%

In 30 days:
30 × 24 × 60 = 43,200 minutes total
0.1% × 43,200 = 43.2 minutes of allowed downtime/month

In 1 year:
365 × 24 × 60 = 525,600 minutes
0.1% × 525,600 = 525.6 minutes ≈ 8.76 hours/year
```

Reliability tiers: | Nines | Availability | Annual Downtime | |---|---|---| | 99% (two nines) | 99% | 87.6 hours | | 99.9% (three nines) | 99.9% | 8.76 hours | | 99.99% (four nines) | 99.99% | 52.6 minutes | | 99.999% (five nines) | 99.999% | 5.26 minutes |

Error budget policy: - Budget full → teams can deploy freely. - Budget 50% consumed → tighten change review. - Budget exhausted → freeze non-critical deployments; all hands on reliability.

Interview Takeaway: SLO is your internal target. SLA is your external promise. Error budget = 100% - SLO; it's how much you can mess up. When budget is gone, stop shipping features.

Cloud Service Models



Model	Control	Responsibility	Flexibility	Time-to-value
IaaS	High	You manage OS, runtime, scaling	High	Slower
PaaS	Medium	You manage app + data	Medium	Fast
SaaS	Low	Use as-is	Low	Instant
FaaS (Serverless)	Minimal	Just write functions	Very Low	Fastest

FaaS (Serverless): AWS Lambda, GCP Cloud Functions. Pay per invocation. Auto-scales to zero. Cold start latency. Great for event-driven, bursty workloads.

SECTION 05

05 Architecture / Diagrams

Container vs VM — Side by Side

VMS VS CONTAINERS — ISOLATION MODEL

VIRTUAL MACHINES	CONTAINERS
App A	App A + Libs — namespace · cgroup
Guest OS (Linux)	App B + Libs — namespace · cgroup
App B	App C + Libs — namespace · cgroup
Guest OS (Windows)	Host OS Kernel
Hypervisor (KVM)	Physical Hardware
Physical Hardware	
GBs · minutes to boot · full isolation	MBs · milliseconds to start · shared kernel

Containers share the host kernel via namespaces & cgroups — faster & denser, but weaker isolation than full VMs.

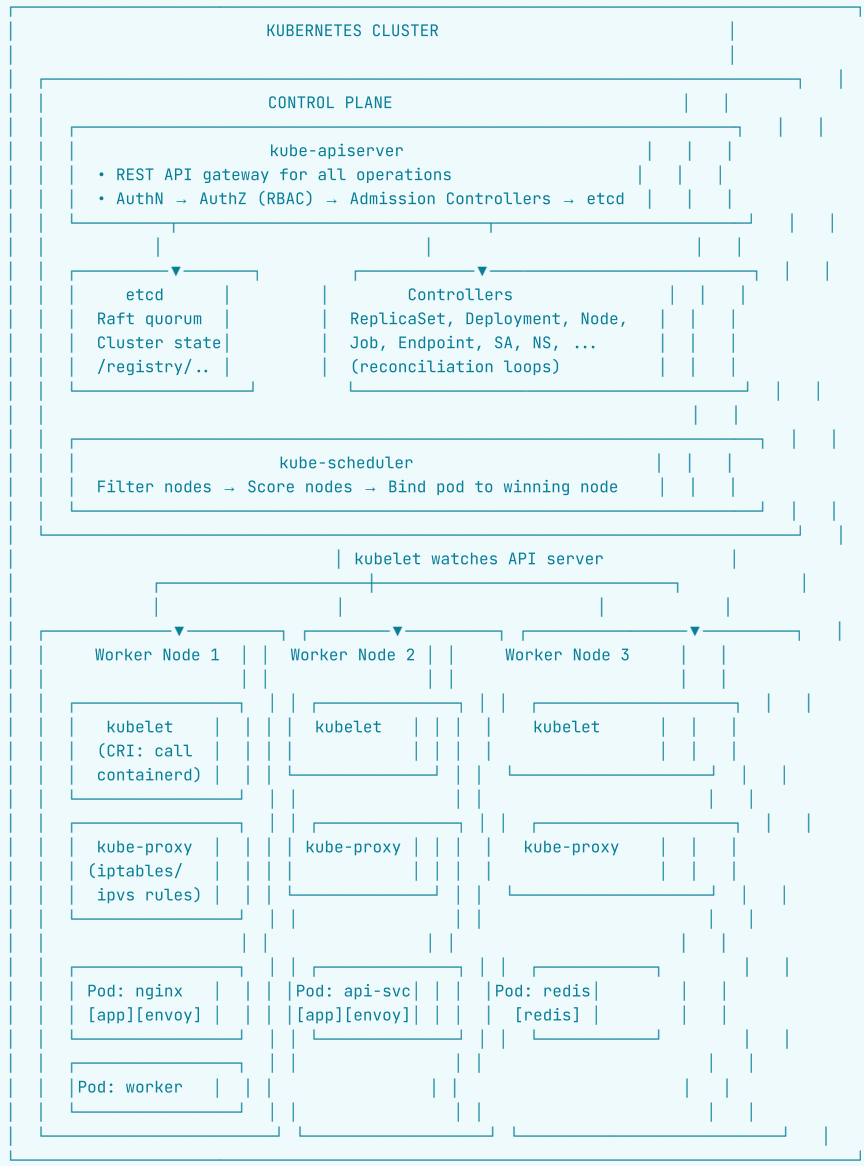
Docker Image Layers

DIAGRAM

```
docker pull python:3.11-slim
docker build -t myapp .

LAYER CACHE:
┌──────────────────────────────────────────────────────┐
│  STEP 5: COPY . .  (app code)     ← changes every build│
├──────────────────────────────────────────────────────┤
│  STEP 4: RUN pip install          ← changes when req.txt changes│
├──────────────────────────────────────────────────────┤
│  STEP 3: COPY requirements.txt    ← infrequent│
├──────────────────────────────────────────────────────┤
│  STEP 2: WORKDIR /app             ← rarely changes│
├──────────────────────────────────────────────────────┤
│  STEP 1: FROM python:3.11-slim    ← stable base, shared on disk│
└──────────────────────────────────────────────────────┘

OverlayFS at runtime:
merged  ← container sees this unified view
  ↑
upperdir (rw) ← container writes here
  ↑
lower4 (ro) ← COPY . .
lower3 (ro) ← RUN pip install
lower2 (ro) ← COPY requirements.txt
lower1 (ro) ← FROM python:3.11-slim
```

Kubernetes Cluster Architecture (Detailed)

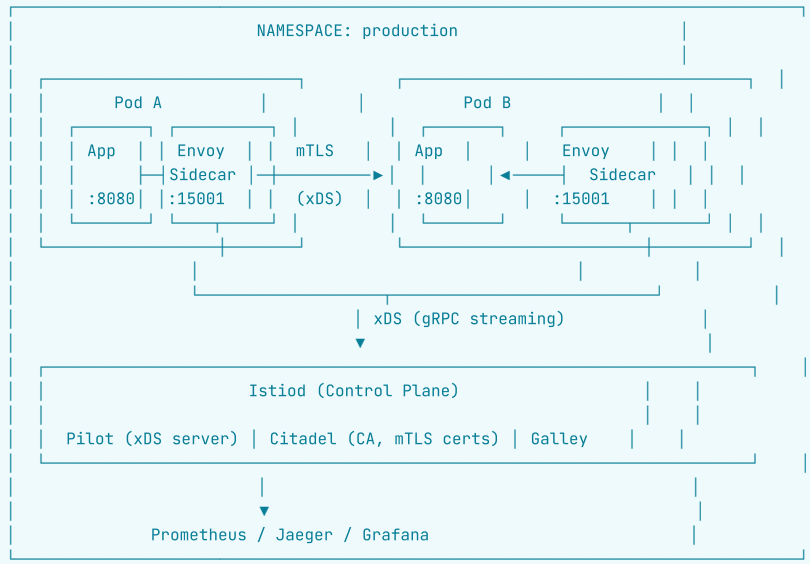
DIAGRAM



DNS: CoreDNS running as pods in kube-system namespace
Network: CNI plugin (Calico, Flannel, Cilium) sets up pod networking
Storage: CSI plugin (EBS, GCE PD) provisions PersistentVolumes

Service Mesh Sidecar Architecture

DIAGRAM




```
Traffic flow (inbound to Pod B):
App → iptables redirect → Envoy sidecar (:15006)
                        → mTLS verify, authz check
                        → route to app (:8080)
                        → emit metric/span
```

CI/CD Pipeline

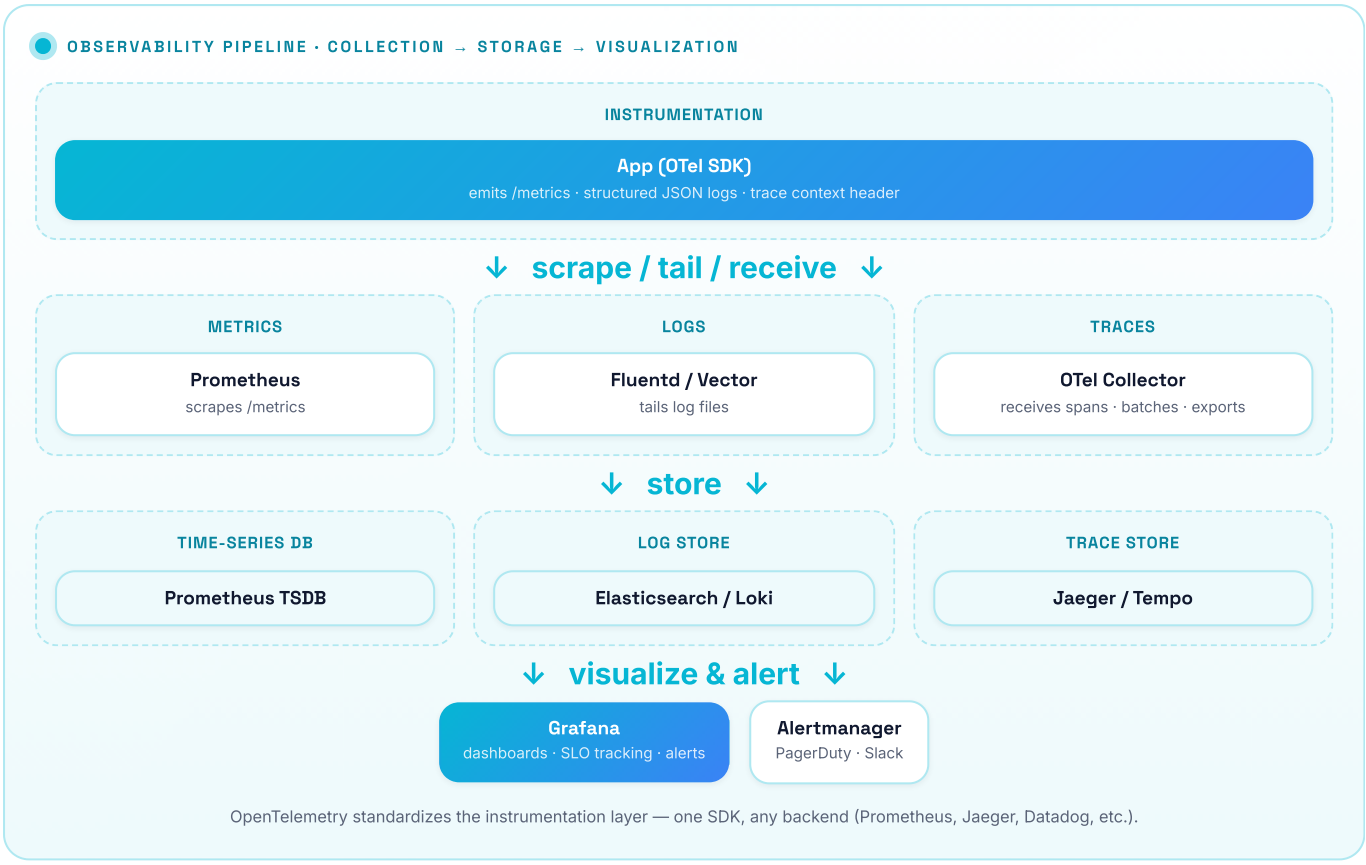
CI/CD PIPELINE · BUILD → TEST → DEPLOY

Developer git push → **GitHub / GitLab** → **CI System** GitHub Actions / Jenkins

CI STAGES	CD STAGES
1. Checkout code	1. Deploy to Staging
2. Lint + SAST (SonarQube, Semgrep)	2. Run smoke / e2e tests
3. Unit tests (pytest, jest)	**[Manual approval gate]**
4. Build Docker image	4. Canary deploy (5% traffic)
5. Integration tests	5. Monitor SLOs for 30 min
6. Security scan (Trivy, Snyk)	6. Expand to 100% or rollback
7. Push image → Registry (ECR/GCR) · tag: sha-abc1234	

Artifact (image:sha-abc) flows from CI → CD; the sha tag ensures immutable, reproducible deployments.

Observability Pipeline


OBSERVABILITY PIPELINE · COLLECTION → STORAGE → VISUALIZATION
INSTRUMENTATION
App (OTel SDK)
emits /metrics · structured JSON logs · trace context header
scrape / tail / receive
METRICS
Prometheus
scrapes /metrics
LOGS
Fluentd / Vector
tails log files
TRACES
OTel Collector
receives spans · batches · exports
store
TIME-SERIES DB
Prometheus TSDB
LOG STORE
Elasticsearch / Loki
TRACE STORE
Jaeger / Tempo
visualize & alert
Grafana
dashboards · SLO tracking · alerts
Alertmanager
PagerDuty · Slack
OpenTelemetry standardizes the instrumentation layer — one SDK, any backend (Prometheus, Jaeger, Datadog, etc.).

SECTION 06

06 Real-World Examples

Google: Borg → Kubernetes

Google ran Borg (container orchestration) internally since ~2003. When containerizing Gmail, Search, and Maps, they packed 2–3 tasks per machine (vs 1 VM per machine), cutting infrastructure costs by 50–70%. In 2014, Google open-sourced Kubernetes as a reimagined, user-friendly Borg. By commoditizing container orchestration, Google leveled AWS's early head start in cloud infrastructure.

Netflix: Chaos Engineering

Netflix runs 100% on AWS. They operate 700+ microservices. To build confidence in their distributed system's resilience, they created Chaos Monkey — a tool that randomly terminates production EC2 instances during business hours. If you can't handle random failures in prod, you'll panic at 3am. Chaos Monkey evolved into the Simian Army (Chaos Gorilla terminates availability zones, Latency Monkey injects delays).

Amazon Prime Video: Microservices → Monolith (2023)

Prime Video moved a monitoring system from serverless microservices (distributed Step Functions + S3) back to a single monolith. Cost dropped 90%. **Lesson:** Microservices aren't always the answer — they add distributed systems complexity and cost. Use them when you need independent scaling or team autonomy.

Shopify: Kubernetes Auto-Scaling on Black Friday

Shopify uses Kubernetes HPA + Cluster Autoscaler to handle 10–20× traffic spikes on Black Friday. They pre-scale (manually set replicas higher before the event) AND rely on HPA for real-time response. Their load tests simulate peak traffic weeks in advance, using observability (Prometheus + Grafana) to catch bottlenecks.

Stripe: Canary Deployments

Stripe deploys hundreds of times per day. Every production deploy goes through a multi-stage canary: 0.1% → 1% → 10% → 100%. Each stage has automatic metric analysis (error rate, latency) with automated rollback. This allows fast iteration without catastrophic incidents.

SECTION 07

07 Real-Life Analogies

One shipping port — every concept is a container, a crane, or a berth in the same terminal.

Concept	Analogy
Container	A standard ISO shipping container: same corner fittings, same locking pins, carries anything from bananas to electronics, slots onto any ship, truck, or rail car without modification — Docker stamps them out, Kubernetes is the port authority that decides where each one goes
Kubernetes scheduler	The port's yard planner: when a new container arrives without a berth, the planner checks every quay for spare capacity, weight limits, and hazmat restrictions (affinity/taints), scores the candidates, then writes the berth number on the manifest — that written assignment is `nodeName` on the pod spec
Namespaces	Sealed container interiors: containers next to each other on the same ship share the hull (host kernel) but each has its own internal atmosphere, its own manifest ID (PID namespace), its own address label (net namespace), its own locked door (mount namespace) — neighbours can't peek inside
cgroups	The port's weight-and-power allocation board: each container slot on a vessel is assigned a maximum gross weight and a shore-power amperage cap; exceed the cap and the breaker trips (OOMKill) or the crane throttles the lift speed (CPU throttling) — one overloaded box cannot capsize the whole ship
OverlayFS / image layers	Stacked pallets inside a container: the bottom pallet is the base OS (read-only), the next is installed dependencies (read-only), the top is your app code; when a container needs to modify a file from a lower pallet, the terminal copies just that item to a new top sheet (copy-on-write) and edits there — the original pallets stay pristine for every other container sharing them
etcd	The harbour master's logbook: every ship, every berth assignment, every container manifest is recorded here using Raft consensus so no single page can go missing; lose the logbook and the port forgets every vessel in the water, every crane allocation, every scheduled departure — the terminal grinds to a halt
HPA	The terminal's shift-rostering desk: when crane throughput data shows containers piling up faster than handlers can move them, the roster desk calls in extra gangs (pod replicas); when the quay clears, surplus gangs are stood down — always targeting the throughput the terminal manager set
Cluster Autoscaler	The port expansion office: when every berth is full and new ships are queuing at the harbour mouth (pending pods), the office requisitions a new quay section from the landlord (cloud API) and opens it; when half the quays sit empty for long enough, it decommissions a section to cut leasing costs
Service Mesh	Port escorts assigned to every container movement: before any box moves between zones, an escort checks the container's RFID seal (mTLS identity), logs the checkpoint in the manifest system (distributed trace span), verifies the receiving zone has clearance (authz policy), and shadows the crane lift — same procedure regardless of which shipping line's logo is on the box
Canary deployment	Trialling a new crane model on one berth first: the operations manager routes a single container down the new crane line; if it lands cleanly within tolerances, the rest of the day's vessels follow; if the new crane drops a box or jams, only that one trial shipment is lost and the old cranes keep running
Blue-Green	Two identical deep-water berths kept in parallel: ships unload at Berth Blue while the operations team fully fits out Berth Green with the new handling equipment; once Green passes inspection, the harbour master flips the gate sign — all incoming vessels now dock at Green; Blue sits ready for five minutes so any problem means flipping the sign straight back
Error Budget	The port's annual slot-delay allowance: the terminal promises carriers that no more than 0.1% of container movements will miss their time window; every incident eats into that allowance; when the allowance is exhausted the port authority freezes all non-essential crane reconfigurations until the next period resets the clock
SLI/SLO/SLA	Quay throughput gauge (SLI) / the terminal's internal target of 400 moves per crane-hour (SLO) / the carrier contract that docks the port's fee by 10% if the target is missed for the month (SLA) — the gauge measures, the target disciplines operations, the contract has teeth
Prometheus pull	The port inspector's rounds: every 15 seconds the inspector walks to each crane, checks the load-cycle counter, and writes it in the notebook — the cranes don't have to radio in; if a crane goes dark the inspector notices immediately because no reading is itself a signal
Distributed trace	The container's full journey log: the gate scanner stamps arrival time (Span 1), the yard tractor logs pickup (Span 2), the crane logs lift-start and set-down (Span 3), the ship's computer logs stow position (Span 4); every leg shares the same bill-of-lading number (Trace ID) so investigators can reconstruct the exact path and find where the two-hour delay occurred
IaC (Terraform)	The terminal's master construction blueprint: instead of building each berth, crane, and road by hand, the harbour engineer submits the blueprint to the construction system which provisions an identical layout on demand — `terraform plan` shows which berths will change before a single pile is driven, and the same blueprint can stamp out a second terminal in a different harbour without starting from scratch

SECTION 08

08 Memory Tricks / Mnemonics

Kubernetes Components: ”EAS-CSK”

Etcd, **A**PI-server, **S**cheduler, **C**ontroller Manager, **S**cheduler (worker: **K**ubelet, kube-proxy)

Control plane: **"EACH Controller Schedules"** - **E**tcd - **A**PI server - **C**ontroller Manager - **H**orizontal scaler (HPA lives here conceptually) - **S**cheduler

Namespace types: ”PUMNIC+T”

PID, **U**TS, **M**ount, **N**et, **I**PC, **C**group, **T**ime (user)

Three Pillars of Observability: ”MLT”

Metrics (what happened), **L**ogs (why), **T**races (where)

Or: **"Every alert needs MLT"** — Metrics alert → Logs for why → Traces for where

Deployment Strategies: ”BCR-R”

Blue-green (instant cutover), **C**anary (gradual real traffic), **R**olling (in-place), **R**ecreate (with downtime)

Remember by risk level: Recreate = most risky (downtime), Rolling = medium (partial overlap), Canary = low (limited blast radius), Blue-Green = safest rollback.

SLI/SLO/SLA: ”Indicator → Objective → Agreement”

I measure the **Indicator** → I set an **Objective** → I sign an **Agreement** with the customer. Error budget = SLO headroom = "how much you can mess up."

Service Types K8s: ”CLNE-H”

ClusterIP → **L**oadBalancer → **N**odePort → **E**xternalName → **H**eadless Or remember by audience: Cluster=internal, LoadBalancer=external-cloud, NodePort=external-bare-metal.

Docker Image Layer Order: ”FROM → WORK → COPY-deps → RUN → COPY-app”

Stable → Volatile (put things that change least at top of Dockerfile).

Metrics types: ”CGH-S” (Counter, Gauge, Histogram, Summary)

- **C**ounters count (never go down)
- **G**auges gauge current state (go up/down)
- **H**istograms have buckets (for percentiles)
- **S**ummaries are summarized (pre-computed, less flexible — avoid)

IaC: ”T = Terra(multi) → CFN = Amazon-native”

Terraform = multi-cloud, HCL. **C**loudFormation = AWS-only, JSON/YAML. "**T**erraform **T**ravels everywhere; **C**loudFormation **C**an't leave AWS."

SECTION 09

09 Common Interview Questions

Q1: What is the difference between a container and a virtual machine?

Model Answer: Containers and VMs both isolate workloads, but they differ in depth of isolation and overhead. A VM runs a full guest OS on top of a hypervisor, which virtualizes hardware. Each VM has its own kernel, so isolation is strong but overhead is high — GBs of image, seconds to boot, significant memory for the guest OS.

A container shares the host kernel but isolates processes using Linux namespaces (what processes can see: PID, network, filesystem, hostname) and cgroups (what processes can use: CPU, memory, I/O). The result is near-zero overhead: MB-sized images, millisecond start times.

The tradeoff: containers have weaker isolation. A kernel vulnerability can potentially allow container escape. For multi-tenant environments (running customer code), VMs are often used. Containers shine for microservices where you trust all the workloads.

Follow-up: How does gVisor or Kata Containers address container isolation? - **gVisor:** a Go-based kernel that intercepts syscalls in a sandbox (runsc OCI runtime) — container kernel isolation without full VM. - **Kata Containers:** lightweight VMs with OCI-compatible interface — VM isolation with container experience.

Q2: Walk me through how Kubernetes schedules a pod.

Model Answer: When you create a pod, it starts with `nodeName: ""`. The kube-scheduler watches the API server for unscheduled pods.

Scheduling happens in two phases:

Filtering: eliminate nodes that can't run the pod. Checks: node has enough CPU/memory (matching pod's `requests`), node selector labels match, pod tolerates all node taints, affinity/anti-affinity rules are satisfied, volumes can be attached.

Scoring: rank remaining nodes. Criteria: LeastAllocated (prefer less-used nodes), BalancedResourceAllocation (CPU vs memory balance), InterPodAffinityPriority (co-locate or spread pods), ImageLocality (node already has the container image).

The scheduler picks the highest-scoring node and writes `nodeName` back to the pod spec via the API server. The kubelet on that node then sees the pod spec, calls the container runtime via CRI, and starts the containers.

Follow-up: What happens if no node has enough resources? - Pod stays in `Pending` state. Cluster Autoscaler detects unschedulable pods and provisions a new node. Once the node registers, the scheduler places the pod.

Q3: Explain the difference between a Deployment and a StatefulSet.

Model Answer: Both manage sets of pods, but they handle pod identity and storage differently.

A **Deployment** manages stateless pods. Pods are interchangeable: if pod-xyz dies, a replacement gets a new random name and IP. Scaling is fast. Rolling updates replace pods one by one. Great for web servers, API services, workers.

A **StatefulSet** manages stateful pods. It guarantees: stable pod names (pod-0, pod-1, pod-2), stable DNS hostnames (pod-0.service.ns.svc.cluster.local), ordered startup (pod-0 ready before pod-1 starts), and each pod gets its own PersistentVolumeClaim that survives pod restarts. This is essential for databases (MySQL, Cassandra), message brokers (Kafka), and coordination services (ZooKeeper).

Follow-up: How does Cassandra work with StatefulSets? - Each Cassandra pod needs a unique identity (node name), stable storage (SSTable files), and stable network address (for gossip protocol between nodes). StatefulSet provides all three.

Q4: How does a Service Mesh work? Why use it instead of client-side libraries?

Model Answer: A service mesh moves cross-cutting concerns (retries, timeouts, circuit breaking, mTLS, observability) out of application code into the infrastructure layer, using sidecar proxies.

In Istio, an Envoy sidecar is injected alongside every application pod. Istio installs iptables rules that redirect all pod traffic through Envoy. The application thinks it's making a direct call; Envoy actually handles the connection. Istiod (control plane) configures all Envoys via the xDS protocol — sending listeners, routes, cluster configurations over gRPC streams.

Benefits over client libraries: language-agnostic (Envoy handles polyglot services), consistent policy (enforced at infra level, not at "did the team update their library?"), zero code changes for observability (traces and metrics auto-emitted from Envoy).

Downsides: latency overhead per hop (~1-5ms), operational complexity, debugging mesh behavior is hard.

Follow-up: What is mTLS and how does Istio implement it? - mTLS = mutual TLS: both client and server authenticate each other with certificates. Istiod acts as a certificate authority, issuing X.509 certs with SPIFFE IDs (e.g., `spiffe://cluster.local/ns/default/sa/my-service`) to each workload. Envoy uses these certs for mTLS handshakes transparently.

Q5: Explain canary vs blue-green deployments. When would you choose each?

Model Answer: Both enable zero-downtime deployments, but differ in how traffic is shifted.

Blue-Green: two identical environments (blue=current, green=new). Deploy to green, test it, then flip 100% of traffic to green instantly (load balancer cutover). Old blue stays live for instant rollback. Cost: 2× infrastructure while both are live.

Choose blue-green when: you need instant rollback capability, the change is all-or-nothing (database schema migration where you can't run two schema versions simultaneously), or you want to test with zero production traffic.

Canary: Route a small percentage (1–5%) of traffic to the new version while the rest goes to stable. Gradually increase if metrics look good. Rollback = reduce canary weight to 0%.

Choose canary when: you want to validate with real production traffic, you can tolerate some users on the new version, and you have good SLO monitoring to auto-detect regressions.

For FAANG-scale services: canary is the dominant strategy because you can't just "test everything in staging" — production traffic has characteristics staging can't replicate.

Follow-up: How does Kubernetes implement canary? - Via multiple Deployments with different labels + weighted Ingress (nginx, Istio VirtualService with weight). Istio makes it cleanest: `weight: 10` on v2 subset, `weight: 90` on v1.

Q6: What are the three pillars of observability? How do they relate?

Model Answer: Metrics, Logs, and Traces are the three pillars.

Metrics are aggregated numerical time-series. They tell you WHAT is happening: CPU at 95%, error rate at 0.5%, p99 latency at 800ms. They're cheap (pre-aggregated), good for alerting, and SLO tracking. But they lose detail — a spike in error rate doesn't tell you which requests failed or why.

Logs are timestamped event records. They tell you WHY: the exact exception, request parameters, stack trace. They're expensive at scale (terabytes/day at FAANG) and hard to query across distributed systems. Structured (JSON) logs are queryable; unstructured text logs are painful.

Traces show WHERE time was spent across service calls. A trace has a TraceID shared across all services; each service creates a Span with timing. They reveal that the 800ms latency is because the payment service is waiting 600ms for a database call on a cold connection pool.

In practice: Prometheus alerts on high error rate (metric) → engineer looks at Elasticsearch logs for failing requests (log) → finds a TraceID in the log → pulls that trace in Jaeger to see exactly which downstream service is broken (trace). They're complementary — each answers a different question.

Q7: What are SLOs and error budgets? How do you use them?

Model Answer: An SLO (Service Level Objective) is an internal reliability target: "99.9% of requests to /checkout will complete in <200ms over a rolling 30-day window." The SLI (Service Level Indicator) is the measurement: actual p99 latency ratio.

The error budget is the allowed amount of failure: 100% - 99.9% = 0.1%. For a 30-day window: 0.1% × 43,200 minutes = 43.2 minutes of budget.

Error budgets make reliability vs velocity tradeoffs explicit. When the budget is full, teams can ship fast. When it's depleted, the SRE policy kicks in: freeze non-critical deployments, cancel planned changes, focus all engineering on reliability.

The power: it depoliticizes "can we deploy?" — it's no longer dev vs ops, it's math. Budget is fine → yes. Budget is gone → no.

Follow-up: What's the difference between SLO and SLA? - SLO is an internal target you're engineering toward. SLA is the external contract with customers, typically with financial penalties. SLAs are usually set lower than SLOs (SLO=99.9%, SLA=99.5%) so you have a buffer.

Q8: How does Terraform manage state? What problems can arise?

Model Answer: Terraform maintains a state file (`terraform.tfstate`) that maps your HCL resource declarations to real cloud resources. This lets Terraform know what it created, what's changed, and what needs to be destroyed.

The state file contains resource IDs, attributes, and dependencies. It's used to calculate diffs: `terraform plan` compares state file vs your desired config vs actual cloud state.

Problems with state: 1. **Concurrent modification:** two engineers run `terraform apply` simultaneously → race condition, corrupted state. Solution: remote state with locking (S3 + DynamoDB lock table). 2. **State drift:** someone manually modifies a resource in the console → state file no longer reflects reality. `terraform plan` shows unexpected changes. Solution: `terraform import`, use `terraform refresh`. 3. **Sensitive data in state:** database passwords, private keys are stored plaintext in state. Solution: encrypt state bucket, use secrets manager for sensitive values. 4. **Large state files:** monolithic repos with all infrastructure in one state file → slow plans, high blast radius. Solution: split into smaller modules/workspaces.

Follow-up: Terraform vs CloudFormation vs Pulumi? - CloudFormation: AWS-native, AWS manages state, automatic rollback on failure, verbose JSON/YAML. Terraform: multi-cloud, you manage state, HCL is cleaner, huge module ecosystem. Pulumi: use TypeScript/Python/Go — good when complex conditional logic is needed.

SECTION 10

10 Senior-Level Discussion Points

Kubernetes at Scale: What Actually Breaks

etcd performance: At 5000 nodes with 50 pods each = 250,000 pod objects. etcd can become a bottleneck. Kubernetes API server uses watch caching (API server caches etcd data; clients watch API server, not etcd directly). Still: large clusters need etcd performance tuning (SSDs, defragmentation, compaction).

Control plane scalability: Kubernetes is tested to ~5,000 nodes and ~150,000 pods per cluster. For larger scales, multiple clusters with federation (Fleet, Argo CD ApplicationSets) is the answer.

Network scalability: kube-proxy with iptables has $O(n^2)$ scaling issues (each service adds iptables rules; 10,000 services = massive iptables traversal). Solution: kube-proxy in IPVS mode (hash table O(1)) or Cilium with eBPF (bypasses iptables entirely, pure BPF maps).

Scheduler throughput: Default scheduler handles ~1000 pod scheduling decisions/second. At hyperscale: custom schedulers, batch scheduling (Volcano), or scheduler extenders.

Service Mesh Overhead and Alternatives

Istio adds ~1-5ms latency per hop and ~10-15% CPU overhead on sidecars. At Netflix scale (billions of requests/day), this is significant.

Alternatives: - **eBPF-based mesh** (Cilium Service Mesh): bypass sidecar overhead entirely by doing L7 in kernel eBPF programs. No sidecar injection needed. - **Proxyless gRPC:** gRPC xDS integration allows direct Istio control plane communication without a sidecar (for gRPC services only). - **Per-service library with common standards:** large orgs like Twitter (Finagle) and Netflix (Ribbon/Hystrix) used fat client libraries before service mesh was mature.

Multi-Cluster and Multi-Region Kubernetes

Why multiple clusters: blast radius containment, regional isolation, team autonomy, different security domains (prod/non-prod), Kubernetes version upgrades (rolling clusters).

Service discovery across clusters: Istio multi-cluster, Submariner, AWS Cloud Map.

GitOps at scale: ArgoCD ApplicationSets generate ArgoCD Applications dynamically for each cluster/environment. One Git repo, many clusters.

Observability: What Interviewers Miss

The cardinality trap: Prometheus stores every unique label combination as a separate time series. Label `user_id` on a metric with 10 million users = 10 million time series → OOM. Never put high-cardinality labels (user IDs, request IDs) on metrics. Use traces for per-request detail.

Head-based vs tail-based sampling: Head-based sampling (decide at trace start) is fast but misses rare slow requests (bad for long-tail latency issues). Tail-based sampling (Grafana Tempo, Honeycomb) collects everything, decides at trace completion — catches all interesting traces but requires buffering all spans (expensive).

Exemplars: Prometheus exemplars link metrics to traces. When a histogram records a high-latency sample, an exemplar stores the TraceID for that specific request. In Grafana: click the high-latency bar in the histogram → jump directly to the trace. This closes the metrics-to-traces gap.

Platform Engineering vs DevOps vs SRE

Role	Focus
SRE (Google model)	Define SLOs, manage error budgets, reduce toil with software, on-call for critical services
DevOps	Culture + practices: break dev/ops silos, automate deployments, shared ownership
Platform Engineering	Build internal developer platforms (IDP) — golden paths for deploying, observing, and operating services; abstracts K8s complexity from product engineers

Platform engineering is the evolved, product-thinking version of DevOps — treating internal developers as customers.

Cost Optimization in K8s

Right-sizing: Most teams over-provision resource requests (fear of OOM kills). VPA in recommendation mode (not enforcement) gives data-driven suggestions without forcing restarts.

Spot/Preemptible instances: Run stateless workloads (batch, ML training) on spot instances (60–90% cost reduction). Use node pool taints/tolerations to route appropriate workloads. Design apps to handle termination signals gracefully (SIGTERM → drain in-flight requests → exit).

Cluster bin-packing: Kubernetes default scheduler doesn't optimize for cost; it spreads load (LeastAllocated). Descheduler (a separate K8s component) can rebalance pods onto fewer nodes during off-peak, enabling cluster autoscaler to scale down idle nodes.

SECTION 11

11 Typical Mistakes Candidates Make

Mistake 1: Confusing Docker with Kubernetes

"I use Docker to run Kubernetes" — wrong. Docker is a container runtime (one of many). Kubernetes uses containerd (or CRI-O), not Docker directly, since K8s 1.24 deprecated the dockershim. You can build Docker images and run them on Kubernetes, but they're separate layers.

Mistake 2: Not Knowing etcd's Role

Many candidates say "Kubernetes stores state in its database" without knowing it's etcd, or that it's a Raft-based distributed key-value store, or that it's the ONLY component kubelets don't talk to directly (all through API server).

Mistake 3: Conflating Secrets with Security

"Kubernetes Secrets are encrypted" — wrong. They're base64-encoded by default. Encryption at rest requires explicit `EncryptionConfiguration` on the API server. Production systems use external secret managers (Vault, AWS Secrets Manager with External Secrets Operator).

Mistake 4: HPA + VPA on Same Metric Conflict

Using HPA and VPA both targeting CPU/memory on the same Deployment causes conflicts. HPA scales replicas based on per-pod CPU; VPA adjusts per-pod CPU requests. Use HPA for scaling, VPA for right-sizing, and don't target the same resource metric with both.

Mistake 5: High-Cardinality Prometheus Labels

Adding user_id, request_id, or other high-cardinality labels to Prometheus metrics causes a time-series explosion (cardinality bomb) that OOMs your Prometheus. For per-request data, use traces.

Mistake 6: Forgetting Resource Requests in Pods

Running pods without `resources.requests` means the scheduler places pods on nodes without knowing actual needs. Result: noisy neighbor problems, OOM kills, unpredictable performance. Always set requests.

Mistake 7: Misunderstanding Rolling Update Safety

A rolling update with `maxUnavailable: 0, maxSurge: 1` ensures no downtime but takes longer. Many candidates don't realize that if the new pods crash immediately, the rollout stalls — it won't remove old pods if new ones aren't ready. They assume rolling updates auto-rollback; they don't (you need `kubectl rollout undo` or a GitOps tool).

Mistake 8: SLO = SLA = Uptime = 99.9%

SLO is internal, SLA is external. SLIs aren't just availability — they can be latency, throughput, correctness. Error budgets aren't just "downtime minutes" — they're burned by any SLO violation (high latency counts). Getting these distinctions right signals SRE maturity.

Mistake 9: IaC Doesn't Mean No Drift

IaC prevents initial drift but doesn't prevent subsequent manual changes. Need: CI enforcement (no console access to prod), Terraform drift detection (`terraform plan` in CI), and possibly tools like Driftctl that detect out-of-band changes.

Mistake 10: Blue-Green Always Better than Canary

Blue-green is simpler and cleaner but costs 2× for the switchover period and doesn't validate with real user traffic. Canary is more complex but validates with real traffic and has precise rollback. The best engineers can articulate the trade-offs for specific scenarios.

SECTION 12

12 How This Connects To Other Topics

System Design

- **Horizontal scaling:** Services designed to be stateless scale horizontally behind K8s HPA. Session state in Redis, not process memory.
- **Database connections at scale:** K8s pods × connections per pod = connection pool exhaustion. Solution: PgBouncer sidecar, connection pool service (RDS Proxy).
- **Service discovery:** In K8s, services use DNS-based discovery (kube-dns/CoreDNS). ClusterIP = stable virtual IP. Headless services for stateful sets: client gets pod IPs directly, enabling consistent hashing.
- **Load balancing:** kube-proxy (L4), Ingress (L7), Istio (L7 with header-based routing). FAANG question: what happens when you need sticky sessions? → use consistent hash in Istio or application-level session affinity.

Distributed Systems

- **etcd = Raft consensus:** etcd is a real-world Raft implementation. Understanding Raft (leader election, log replication, quorum) helps answer "what happens if 2 of 3 etcd nodes fail?" (cluster becomes unavailable; no split-brain because writes need quorum).
- **Eventual consistency:** K8s is eventually consistent. The controller's reconciliation loop retries until actual state matches desired state. Pods may take seconds to schedule, containers to start, services to register.
- **CAP theorem:** During a network partition, K8s chooses CP (consistency + partition tolerance) — the API server returns errors rather than serving stale data if it can't reach etcd quorum.
- **Idempotency:** K8s controllers are designed to be idempotent — running the same reconciliation loop twice produces the same result. This enables safe crash recovery.

Networking

- **Container networking (CNI):** Kubernetes delegates pod networking to CNI plugins. Each pod gets a unique IP routable within the cluster. CNI plugins: Flannel (simple overlay, VXLAN), Calico (L3 routing, BGP, network policy), Cilium (eBPF, L7 policies, no iptables).
- **kube-proxy and iptables:** ClusterIP services work via DNAT iptables rules. Understanding netfilter/iptables is essential for debugging service connectivity.
- **Ingress and TLS:** TLS termination at Ingress (nginx, Traefik) or Istio Gateway. Understanding TLS handshake, cert management (cert-manager + Let's Encrypt), and SNI routing.
- **Network policies:** K8s NetworkPolicy resources control pod-to-pod traffic (default: allow all). Enforced by CNI plugin (Calico, Cilium). Zero-trust network: default deny, explicit allow.

Performance

- **Cold starts:** Container startup latency = image pull (if not cached) + runtime init + app startup. For latency-sensitive services: pre-pull images on nodes (DaemonSet), use slim base images, preload app dependencies.
- **CPU throttling:** Setting CPU limits causes CFS throttling — your container gets CPU in 100ms windows proportional to its quota. A 250m limit = 25ms of CPU per 100ms window. Under bursty load, this causes latency spikes. Many FAANG teams remove CPU limits but keep requests for scheduling.
- **Memory and OOM:** Memory limit breach = OOMKill (immediate SIGKILL, no graceful shutdown). Size your limits with headroom. Use Prometheus to track `container_memory_working_set_bytes` and adjust.
- **Resource contention:** Without resource requests, pods can compete for CPU/memory on the same node. Use LimitRanges (namespace-level defaults) and ResourceQuotas (namespace-level ceilings).

SECTION 13

13 FAANG Interview Tips

What Google Looks For (SRE/Infrastructure interviews)

- Deep K8s internals (scheduler, controller loops, reconciliation).
- SLO/error budget thinking — be ready to propose SLIs for any service.
- Borg/K8s history and design philosophy (simplicity, declarative API).
- Operations at scale: what breaks at 10,000 nodes? How do you debug a cluster with 10,000 pods all crashing?

What Meta Looks For

- System design with scale in mind (Facebook has 10B+ daily requests).
- Tradeoffs: when to use containers vs bare metal, when to build vs buy.
- Custom tooling philosophy — Meta builds much of their infra from scratch.
- Deep networking knowledge (BGP, ECMP, spine-leaf topology at data center scale).

What Amazon (AWS) Looks For

- AWS services depth: know ECS vs EKS, CloudFormation vs CDK, ALB vs NLB.
- Operational excellence mindset — every design question should include "how do you operate this?" and "how do you recover from failure?"
- Leadership Principles applied to tech decisions (Bias for Action, Frugality, Operational Excellence).

What Netflix Looks For

- Resilience engineering — chaos engineering mindset.
- Observability-first design — instrument everything from day 1.
- Blast radius minimization — design for failure, not just for the happy path.

Universal FAANG Tips

1. **Lead with "why" before "how."** Don't just describe K8s architecture — explain why each component exists, what problem it solves.
2. **Mention tradeoffs proactively.** "Blue-green is simpler but costs 2×. Canary is more complex but validates with real traffic." This signals senior-level thinking.
3. **Connect to real incidents.** "I've seen this cause..." demonstrates production experience, not just textbook knowledge.
4. **Scale your answers.** When designing, always ask/answer: "What does this look like at 10× current scale? 100×?"
5. **SLO-driven thinking.** For any system, immediately think: what are the SLIs? What would the SLO be? How would you alert on SLO violations?
6. **Know your numbers:** - Kubernetes: ~5000 nodes/cluster supported, ~150k pods - etcd: 8GB recommended memory, 1GB default storage limit for keys - Prometheus: ~1M active series per instance (horizontal sharding for more) - Container cold start: ms; VM boot: 30s-5min - 99.9% uptime = 43.2 min/month downtime
7. **eBPF is the future.** Know that Cilium/eBPF is replacing iptables-based networking and sidecar-based service meshes. Mentioning this at FAANG signals you're tracking the latest tech.
8. **GitOps is default.** For deployment questions, propose ArgoCD/Flux for CD. "We'd store manifests in Git, ArgoCD would sync to the cluster. Any change is auditable and rollback is a git revert."

SECTION 14

14 Revision Cheat Sheet

10-Minute Summary

Containers pack apps with their dependencies (not the OS) using Linux **namespaces** (isolation) and **cgroups** (resource limits). They're faster and lighter than VMs. Docker images are layered via **OverlayFS** with copy-on-write.

Kubernetes orchestrates containers. Control plane = **API server** (entry point) + **etcd** (state store) + **Scheduler** (places pods) + **Controller Manager** (reconciliation loops). Workers run **kubelet** (runs pods) + **kube-proxy** (routes service traffic). K8s is declarative — you state desired state, controllers reconcile.

HPA scales pod count on metrics. **VPA** right-sizes requests. **Cluster Autoscaler** scales node count. Use HPA + CA together; avoid HPA + VPA on same metric.

Service Mesh (Istio/Envoy) handles cross-cutting concerns (mTLS, retries, tracing) via sidecar proxies. App is unaware. Control plane (Istiod) pushes config via xDS.

CI/CD: CI = build + test. CD = deploy. Strategies: **Rolling** (in-place gradual), **Blue-Green** (instant switchover, 2× cost), **Canary** (real traffic validation). FAANG uses canary.

IaC: Terraform = multi-cloud, HCL, you manage state. CloudFormation = AWS-native, AWS manages state.

Observability pillars: Metrics (what) → alert; **Logs** (why) → debug; **Traces** (where) → diagnose latency.

SLO/Error Budget: SLO = internal target. SLA = external contract. Error Budget = 100% - SLO. When budget exhausted → freeze features, fix reliability.

Key Points — The Must-Remembers

- **Namespaces = isolation (what you see); cgroups = limits (what you use)**
- **etcd = brain of K8s; losing it = losing cluster state (back it up!)**
- **Scheduler: Filter → Score → Bind**
- **HPA scales pods. Cluster Autoscaler scales nodes. Don't confuse.**
- **K8s Secrets = base64 (not encrypted). Enable EncryptionConfiguration + use Vault.**
- **Service = stable VIP + DNS in front of dynamic pods (selected by labels)**
- **Canary = real traffic validation; Blue-Green = instant rollback; Rolling = default K8s**
- **Three Pillars: Metrics (what) + Logs (why) + Traces (where)**
- **Counter never goes down; Gauge goes up/down; Histogram has buckets**
- **Error Budget = 100% - SLO; exhausted = freeze deployments**
- **Terraform state = source of truth; store in S3 + lock with DynamoDB**
- **High-cardinality labels kill Prometheus; never label with user_id**
- **eBPF (Cilium) = future of K8s networking; replaces iptables + sidecar mesh**

Cheat Sheet Table

Concept	Key Detail	Interview Quote
Container isolation	Namespaces + cgroups	"Namespace: what you see. cgroup: what you use."
OverlayFS	CoW; layers are read-only; container layer rw	"Image layers are immutable; changes copy to upper layer"
etcd	Raft consensus; only API server talks to it	"Cluster brain; quorum (n/2+1) required for writes"
Scheduler	Filter → Score → Bind	"Find viable nodes, rank them, write nodeName"
HPA	Scales replicas based on metrics	"ceil(current × currentMetric / targetMetric)"
VPA	Adjusts resource requests	"Right-size, requires pod restart; conflicts with HPA"
Cluster Autoscaler	Provisions/removes nodes	"Triggered by unschedulable pods; scales down idle nodes"
K8s Service types	ClusterIP/NodePort/LB/ExternalName	"ClusterIP=internal; LB=cloud external"
Ingress	L7 routing by host/path	"Requires Ingress Controller (nginx, Traefik)"
Secrets	base64, not encrypted by default	"Enable EncryptionConfiguration; use Vault in prod"
StatefulSet	Stable names/DNS/PVC per pod	"For DBs, Kafka, ZooKeeper — ordered deploy"
Service Mesh	Sidecar Envoy; Istiod control plane	"mTLS, retries, traces — zero code changes in app"
Canary	Real traffic subset; auto-rollback on metrics	"1%→5%→25%→100%; abort if error rate spikes"
Blue-Green	Two environments; instant LB flip	"2× cost; instant rollback; good for big-bang changes"
Terraform state	JSON file; S3 + DynamoDB lock	"Sensitive data in state; encrypt bucket"
Prometheus pull	Scrapes /metrics; TSDB; PromQL	"Pull model; Pushgateway for batch jobs"
Counter	Monotonic; use rate() to get rate	"http_requests_total — never subtract"
Histogram	Configurable buckets; use histogram_quantile()	"p99 latency; remember to set appropriate buckets"
Distributed trace	TraceID across services; spans with timing	"Where time went; OpenTelemetry for instrumentation"
SLO	Internal reliability target	"99.9% = 43.2 min/month budget"
Error Budget	100% - SLO	"When exhausted: freeze deploys, fix reliability"
SLA	External contract with penalties	"Usually lower than SLO; SLO=99.9%, SLA=99.5%"
IaaS/PaaS/SaaS	You manage: runtime/app/nothing	"EC2=IaaS, Heroku=PaaS, Gmail=SaaS"
eBPF	Kernel-level networking, replaces iptables	"Cilium: O(1) lookup, no sidecar needed, L7 in kernel"
GitOps	Git = desired state; ArgoCD syncs cluster	"git revert = rollback; git log = deploy history"

The Most Important Concepts (Rank-Ordered for FAANG)

1. **Kubernetes architecture end-to-end** — control plane components, worker node components, and how a pod goes from `kubectl apply` to running. Draw this from memory.
2. **Container internals** — namespaces, cgroups, OverlayFS. "Isolation without a VM" — know the mechanism, not just the definition.
3. **SLO/SLA/Error Budget** — the Google SRE mental model. Every FAANG SRE interview will test this. Know the error budget math.
4. **Deployment strategies** — blue-green vs canary vs rolling with tradeoffs. Know when to use each and how Kubernetes/Istio implements them.
5. **Observability pillars** — metrics/logs/traces: what each answers, when to use each, and how they connect (exemplars, trace correlation in logs).
6. **K8s scheduling and autoscaling** — Filter/Score/Bind, HPA vs VPA vs Cluster Autoscaler, and when they conflict.
7. **Service Mesh** — sidecar pattern, mTLS, xDS protocol. Know why it exists beyond "it's cool."
8. **Infrastructure as Code** — Terraform state management, drift, and remote state. CloudFormation vs Terraform tradeoffs.
9. **Resource requests vs limits** — Requests for scheduling, limits for cgroup enforcement. CPU throttling vs memory OOMKill behavior.

10. **etcd** — Raft consensus, why losing etcd is catastrophic, why only the API server writes to it, backup strategies.

Last updated: 2026-06-14 | Topic: Cloud & Infrastructure | Level: FAANG Senior/Staff

ENGINEERING ESSENTIALS

Master the fundamentals.

Understand the why before the how. Connect every concept to a real system, an analogy, and a trade-off you can defend.

Vol. 04 — Cloud & Infrastructure

FAANG Interview Master Series